Appendix C: IEEE 830 Template

Software Requirements Specification

For

campUStrade

Version 1.4

Prepared by Daniele Conte,Kalliopi Kydonaki

Organisation: Lodz University of Technology

Date created: 17.04.2026

«Page Break»

Table of Contents

«Table of Contents»

Revision History

Name	Date	Reason for Changes	Version
Daniele Conte	17.04.2026	Initial Version, introduction and functionalities	1.0
Daniele Conte	17.04.2026	Adding the “Use Case Diagram” in 2.2 section and modifying the 2.3 section	1.1
Daniele Conte	20.04.2026	Adding the “UML Class Diagram”	1.2
Daniele Conte	20.04.2026	Adding examples ofSequence, State and Activity diagrams	1.3
Kalliopi Kydonaki	27.04.2026	Adjustments for use case diagram and class diagram Completed chapters 1, 2, 3, 4 missing parts	1.4

«Page break»

1. Introduction

1.1. Purpose

The purpose of this document is to specify the software requirements of the “campUStrade” platform and to introduce it: the scope of this document covers the core functionalities necessary for the initial release, specifically focusing on user registration, the creation of service listings, and the management of the virtual credit transaction system between students.

1.2. Document Conventions

«Describe any standards or typographical conventions that were followed when writing this specification, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.»

This document follows a standard Software Requirements Specification structure. Section titles and subtitles are numbered hierarchically in order to make the document easy to read and reference. Functional requirements are identified using the format FR-XX, where XX is a unique number. The word “shall” is used to indicate mandatory system requirements. Priorities are defined for system features as High, Medium, or Low. High-priority requirements are essential for the basic operation of the system, while medium and low-priority requirements describe supporting or optional functionality.

3. Intended Audience and Reading Suggestions

«Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this document contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.»

This document is intended for developers, testers, project managers, system administrators, and academic evaluators. Developers should use this document to understand the system’s required functionality and implementation constraints. Testers should use the functional requirements to create test cases and verify that the system behaves correctly. Project managers can use the document to understand the project scope and priorities. System administrators can refer to the document to understand their role in managing users and resolving platform issues. Readers are advised to begin with Sections 1 and 2, which provide the general purpose, scope, and context of the product. Developers and testers should then focus on Sections 3, 4, and 5, where the main system features, external interfaces, and functional requirements are described in more detail.

1.4. Product Scope

«Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies when applicable.»

University students have to deal with many financial problems, making it difficult to afford professional services such as academic tutoring, laptop repairs or language practice. At the same time, universities are an environment rich in human capital, where thousands of individuals possess diverse and highly valuable skills. The primary problem is the absence of a centralized, secure, and moneyless platform to make easier the exchange of these services among them.

"campUStrade" is a web-based software designed to solve this problem by introducing a virtual credit-based economy. The system allows students to offer their specific skills to earn virtual credits, which they can spend to request services from other students as well.

By removing a real currency, the product aims to foster a collaborative community, making essential services accessible to all students regardless of their financial situation.

1.5. References

«List any other documents or Web addresses to which this document refers or was based on. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location. Include the assignment document in the references.»

The following documents and sources were used or may be referenced during the development of this specification:

Course Assignment Document, provided by the course instructor, 2026.

Software Requirements Specification Template, provided for academic use.

IEEE-style guidelines for Software Requirements Specification documents.

Project notes and team discussions regarding the campUStrade platform concept and functionality.

2. Overall Description

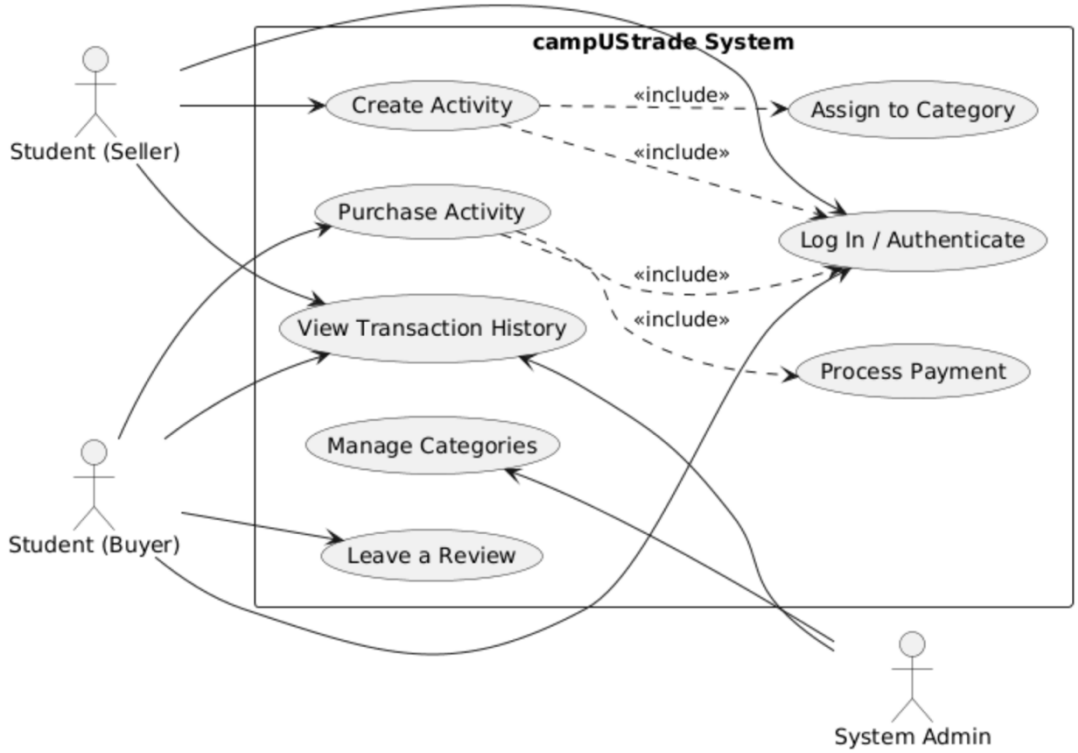
2.1. Product Perspective

«Describe the context and origin of the product being specified in this specification. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product.»

CampUStrade is a new, self-contained web-based application. It is not a replacement for an existing system, but a new platform designed specifically for university students who want to exchange services without using real money. The product will operate as an independent web platform where students can register, publish service offers, book services, and exchange virtual credits. The system introduces a controlled virtual economy inside the university environment, allowing students to benefit from each other’s skills in a secure and organized way.

2.2. Product Features

The campUStrade platform provides several core features to allow a moneyless exchange of services among students. The following Use Case Diagram illustrates these major functionalities and how the users interact with the system:



Summary of the main features:

- Account Registration: Students register using their university email. The system automatically verifies the email and grants a welcome bonus.
- Service Listing: Users can publish offers for their specific skills.
- Service Booking: Users can spend their virtual credits to book services from others, provided they have sufficient funds.
- Service Completion: Upon confirmation of a finished job, the system securely transfers the virtual credits from the buyer to the seller.

2.3. User Classes and Characteristics

The campUStrade platform is designed primarily for one specific user class, with a secondary administrative class for maintenance:

- Student (Primary User Class): The main target audience for this product. These are university students looking to exchange services. They must possess a valid university email address. They interact with all the main features of the system (registering, publishing, booking, and confirming services).
- System Administrator (Secondary User Class): A technical user responsible for monitoring the platform, resolving disputes between students, and maintaining the system. They have high technical expertise and advanced privilege levels to manage users and virtual credits if things go wrong.

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»
The campUStrade system will operate as a web-based application accessible through modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The system is expected to run on standard desktop and laptop computers, as well as mobile devices with internet access. The server side of the application may be hosted on a university server or cloud-based hosting environment. The application will require a database system to store user accounts, service listings, bookings, and virtual credit transactions.

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

The system must be implemented as a web-based application. Users must have internet access and a valid university email address in order to register and use the platform. The system must use a database to store user data, service listings, bookings, and credit balances. Security is an important constraint, since the platform handles user accounts and virtual credit transactions. Therefore, user authentication, email verification, and access control must be implemented. The system should also be designed with a simple and user-friendly interface, because the main users are students with different levels of technical experience. The implementation should follow good programming practices and should be easy to maintain and extend in the future.

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

The following user documentation may be delivered with the software:

A short user manual explaining how students can register, publish a service, book a service, and confirm service completion.

Online help messages or tooltips inside the platform to guide users through the main actions.

A short administrator guide explaining how system administrators can monitor users, resolve disputes, and manage virtual credit issues.

A FAQ section answering common questions about virtual credits, bookings, and account verification.

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

It is assumed that all users are university students and have access to a valid university email address. The system depends on email verification in order to confirm that a user belongs to the university community. It is also assumed that users will honestly confirm the completion of services. However, because disputes may occur, the system depends on the presence of a system administrator who can review problems and take corrective action when necessary. The platform also depends on a stable internet connection, a working database, and a reliable hosting environment. If any of these external factors fail, the system's availability and performance may be affected.

3. System Features

The campUStrade system provides the main functions needed for a virtual credit-based service exchange platform. The system features described in this section are explained in more detail in Section 5.

3.1. User Registration and Email Verification

This feature allows students to create an account using a valid university email address. After successful verification, the system creates the user account and gives the student an initial amount of virtual credits.

3.2. Service Listing Management

This feature allows students to publish services they are willing to offer. Each service listing includes a title, description, and cost in virtual credits.

3.3. Service Booking

This feature allows students to book services offered by other users. The system checks whether the buyer has enough virtual credits before allowing the booking.

3.4. Virtual Credit Transaction

This feature manages the deduction, holding, and transfer of virtual credits. Credits are deducted from the buyer when a service is booked and transferred to the seller only after service completion is confirmed.

3.5. Service Completion Confirmation

This feature allows the buyer to confirm that the requested service has been completed. After confirmation, the system transfers the virtual credits to the seller.

3.6. Administration and Dispute Management

This feature allows the system administrator to monitor the platform, manage users, and resolve problems related to bookings or virtual credit transactions.

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

The system will provide a graphical user interface through a web browser. Users will interact with the platform by clicking buttons, filling in forms, selecting services, and confirming actions.

The main screens of the system will include:

Registration/Login screen: allows students to create an account or log in using their university email.

Home/Dashboard screen: displays the user's virtual credit balance, available services, and quick access to main functions.

Service Listing screen: displays available services with their title, description, seller, and credit cost.

Create Service screen: allows a user to publish a new service by entering a title, description, and price in virtual credits.

Booking screen: allows the user to book a selected service if they have enough credits.

My Bookings screen: displays the services the user has booked and their status.

Service Completion screen: allows the buyer to confirm that the service has been completed.

Administrator screen: allows administrators to manage users, bookings, disputes, and virtual credit issues.

The interface should be simple, clear, and easy to use. Important actions such as booking a service or confirming completion should display confirmation messages. Error messages should be clear and explain what went wrong, for example when a user does not have enough virtual credits.

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

The system does not require any special hardware interfaces. It will operate on standard devices such as desktop computers, laptops, tablets, and smartphones. Users will interact with the system using standard input and output devices, such as a keyboard, mouse, touchscreen, and monitor. The system requires internet access in order to communicate with the server and database.

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

The campUStrade system will interact with the following software components:

- Web browser: used by students and administrators to access the platform.
- Web server: hosts the application and handles user requests.
- Database management system: stores user accounts, service listings, bookings, credit balances, and transaction records.
- Email verification service: used to verify that users register with a valid university email address.

Data exchanged between these components includes registration information, login credentials, service details, booking requests, credit balances, and transaction confirmations. Communication between the client and server should be secure in order to protect user data.

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. User Registration and Email Verification

5.1.1 Description and Priority

This feature allows new users to create an account using a valid university email address. It has high priority because only verified university students should be allowed to access the platform.

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

Stimulus: A student enters a valid university email address and submits the registration form.
Response: The system verifies the email address and creates a new user account.

Stimulus: A student enters an invalid or non-university email address.
Response: The system rejects the registration and displays an error message.

5.1.3 Functional Requirements

«Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.»

FR-01: The system shall allow users to register an account strictly using a valid university email address (e.g., email ending in @edu.p.lodz.pl).

FR-02: The system shall allow a user to publish a service listing containing a title, a text description, and a specific cost in virtual credits.

FR-03: The system shall deduct the specified amount of virtual credits from the buyer's account immediately upon booking a service.

FR-04: The system shall transfer the held virtual credits to the seller's account only after the buyer clicks the "Service Completed" button.

FR-05: The system shall prevent a user from booking any service if their current virtual credit balance is strictly lower than the service's credit cost.

FR-06: The system shall allocate 10 virtual credits to the account balance of every newly registered user upon successful email verification.

6. Nonfunctional Requirements

6.1. Performance

NFR-01: The system shall process and reflect virtual credit transfers between user accounts in less than 1.5 seconds.

NFR-02: The system shall allow a new user to complete the account registration process in less than 2 minutes.

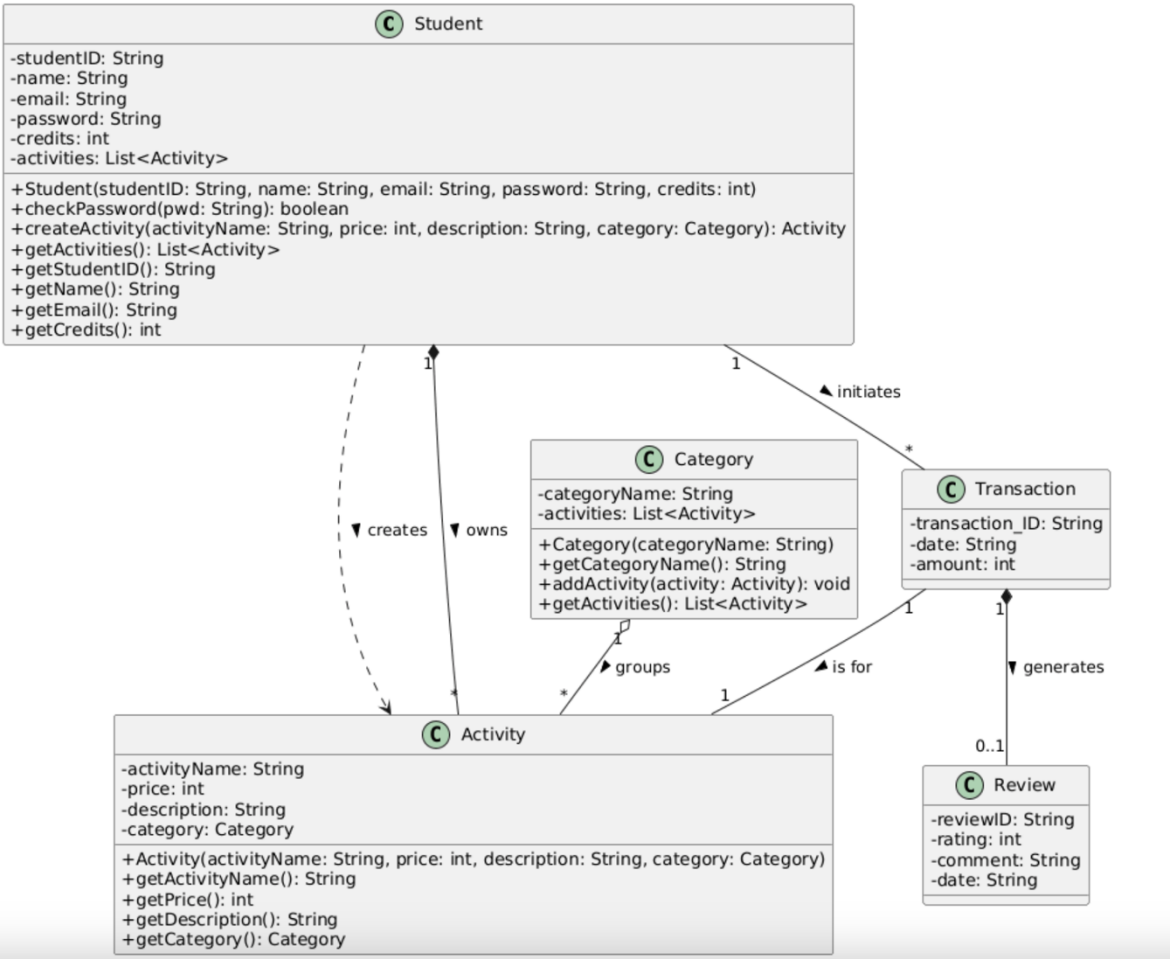
6.2. Security

NFR-03: All user passwords shall be hashed using the bcrypt algorithm before being stored in the database.

7. System Design: UML Class Diagram

7.1 Overview: This section illustrate the data architecture of campUStrade in a UML Class Diagram, which enlights the entities and their private attribute, methods and the relations between them.

7.2 Class Diagram



7.3 Class Description

The system is divided in 5 core classes:

- Student: Represents the system user. It encapsulates the virtual currency balance (-credits) and provides secure methods for financial interactions, such as “+use_credit()” and “+receive_credits()”.
- Activity: Represents a service or task offered by the student (e.g., tutoring for “Database Programming”).
- Category:Logically groups various activities to improve searchability and platform organization.
- TransactionActs as a passive historical record that tracks credit exchanges between a student and a specific activity.
- Review: Manages the feedback system (rating and comments) generated upon the successful completion of a transaction.A student must leave a feedback so it can help other students to choose their activities more wisely.

7.4 Design Decisions & Relationships

To ensure data integrity and proper object lifecycle management, the following relationships were implemented:

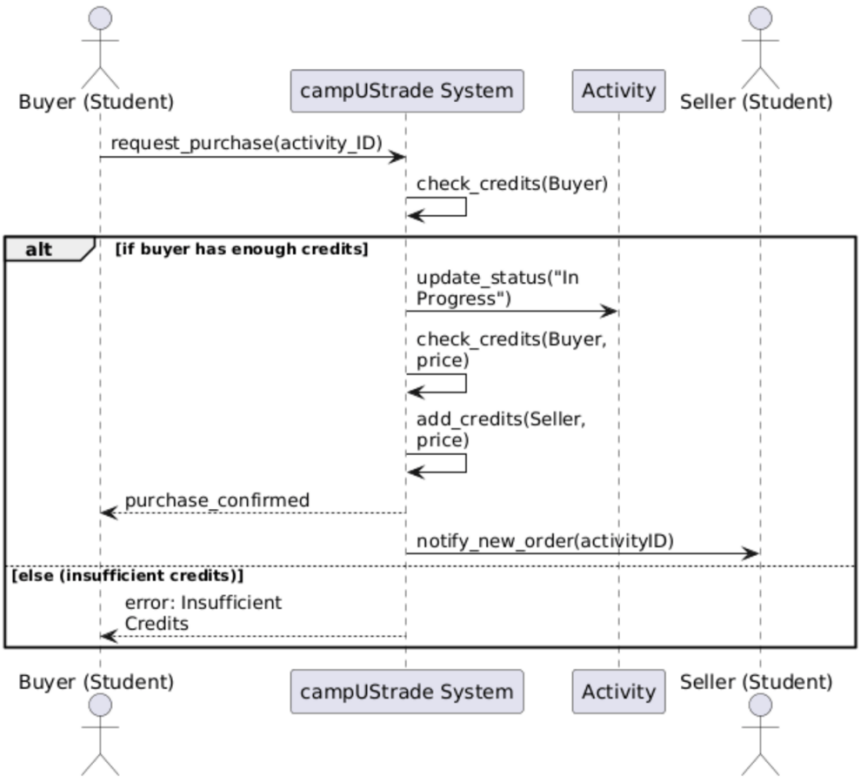
- Composition (Student -- Activity): A strong "part-of" relationship. If a Student account is deleted, all associated Activities are removed to maintain a correct order in the database.
- Composition (Transaction -- Review): A Review cannot exist without a completed Transaction. Deleting the transaction record automatically removes the associated feedback.
- Aggregation (Category o-- Activity): A weak "has-a" relationship. Deleting a Category does not destroy the Activities within it; they are simply unassigned or moved to another category.
- Dependency (Student ..> Activity): The Student class depends on the Activity class specifically for the creation process, as shown in the “+create_activity()” method.

8.0System Behavior& Process Modeling

8.1 Overview

This section defines some of the behavior of the campUStrade system. The following behavioural diagrams (Sequence, State, and Activity) illustrate how objects interact over time, the lifecycle of core entities, and the step-by-step logic of key operations.

8.2 Sequence Diagram: Purchasing an Activity

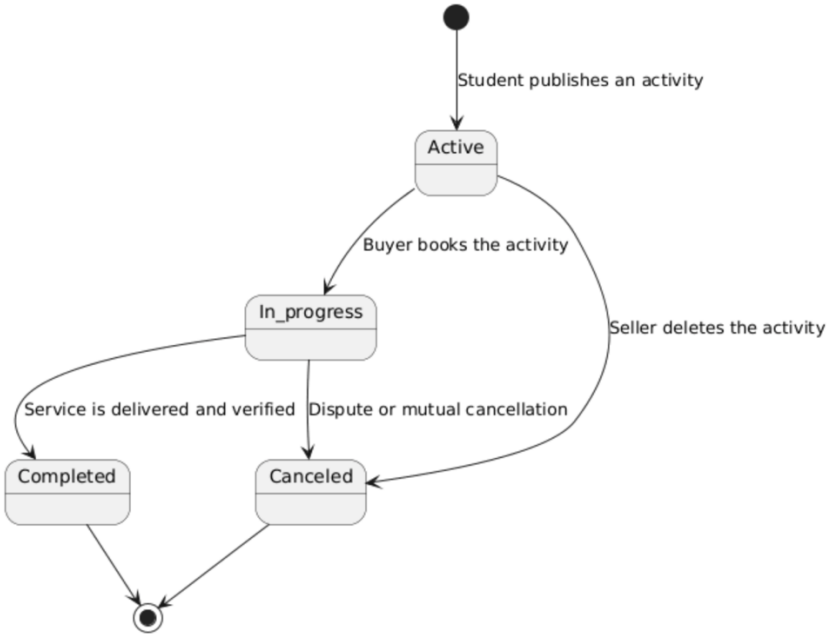


This sequence diagram illustrates the chronological interactions between the Actor (Buyer), the core system, the Activity object, and the Seller when a purchase is initiated.

Synchronous Messages: Used when the system needs to verify data (e.g., “check_credits”) before proceeding.

Combined Fragments (Alt): An alt frame is implemented to handle conditional logic. If the buyer has sufficient virtual credits, the system updates the activity status, processes the transaction, and notifies the seller. If credits are insufficient, an alternative path is executed, returning an error message to the buyer.

8.3 State Diagram: The lifecycle of an activity

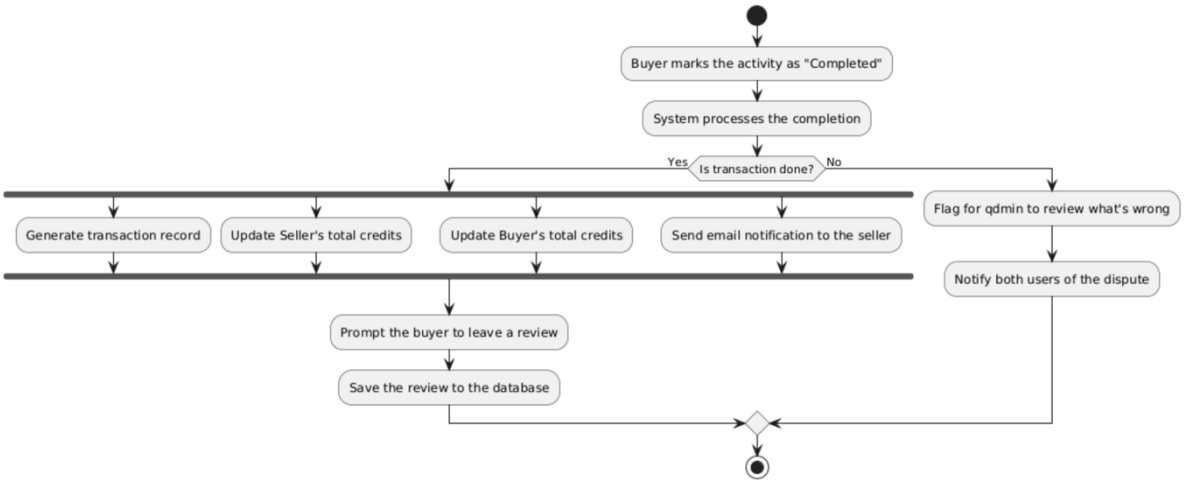


This state diagram maps the lifecycle of an Activity from its creation to its termination within the platform.

States and Triggers: The object begins at the Initial Node and enters the “Active” state upon publication. Specific events such as "Buyer books the activity" or "Seller deletes the activity" cause transitions to other states like “In_progress” or “Canceled”.

Final Nodes: The lifecycle concludes at a Final Node only when the service is Completed or definitively Canceled.

8.4 Activity Diagram: about the transaction



This activity diagram models theexecution when an activity is marked as completed by the buyer.

Decision Nodes (Guards): A diamond decision node evaluates if the completion request is valid. If not, it is routed to an “administrative review” path.

Concurrency (Fork/Join): Updating the seller's credit balance,Updating the buyer's credit balance, generating the historical transaction record and sending notification emails: all executed simultaneously by the system to optimize performance, before finally prompting the buyer to leave a review.